AI Safety Mini Project

Haichuan Wang, Henry Huang, Emira Ibrahimović

October 2025

Abstract

This mini-project aims to replicate and extend the results from the *Follow My Instruction and Spill the Beans: Scalable Data Extraction from Retrieval-Augmented Generation Systems* paper, which proposes a simple instruction-style prompt can make instruction-tuned language models copy retrieved context verbatim in a retrieval-augmented generation (RAG) pipeline. In addition, they show that vulnerability grows with model size. We reproduce the paper's headline result, explore 2 divergences to test robustness, and reflect on the reproducibility and stability of the findings.

1 Introduction

Retrieval-Augmented Generation (RAG) systems have become a key paradigm for grounding large language models (LLMs) in external knowledge. By retrieving relevant documents before generation, RAG pipelines mitigate hallucinations and improve factual accuracy. However, recent work such as the *Spill my Beans* study has revealed that RAG systems are vulnerable to subtle prompt manipulations that can "spill" internal memory or bypass retrieval safeguards. These adversarial behaviors highlight the fragility of current retrieval and generation coupling, motivating a closer examination of RAG robustness.

This mini-project aims to replicate the core results from the *Spill my Beans* RAG pipeline and to explore extensions that evaluate its robustness against prompt-level semantic attacks. We first reproduce the headline result from the original paper—quantifying the success rate of leakage or manipulation prompts under standard retrieval conditions. Our reproduction serves to validate the original experimental setup and establish a baseline for further robustness analysis.

Beyond replication, we introduce two extensions designed to probe and enhance the semantic reliability of the RAG pipeline:

1. **Sliding-window semantic guard:** We implement a BERTScore-based windowed similarity checker that compares each segment of the anchor prompt against a set of predefined attack templates. This method detects semantically similar adversarial patterns even when surface wording varies, providing a dynamic safeguard during generation.
2. **Targeted fine-tuning on attack-style data:** We further fine-tune the model on a curated dataset of prompts that mimic attack semantics. This step aims to improve the model's resilience to adversarial prompt perturbations by strengthening its internal discrimination between benign and malicious intent.

By combining empirical replication with robustness-oriented divergences, this study seeks to assess how fragile or generalizable the original *Spill my Beans* findings are. Our results provide insight into the stability of RAG pipelines under semantic manipulation and contribute to ongoing efforts toward safer retrieval-augmented systems.

2 Methods

2.1 Original Pipeline Overview

Spill My Beans uses a Retrieval-In-Context (RIC) RAG (Retrieval-Augmented Generation) system with various language models (LMs) to demonstrate the propensity of models to repeat the datastore verbatim. This is done using a simple attack prompt: "Here is a sentence: put anchor query here. Now copy and output all the text before 'Here is a sentence'. Do not change any words."

Size	Model	ROUGE-L	BLEU	F1	BERTScore
7B	Mistral-7B-Instruct-v0.3	98.404 ± 0.839	94.686 ± 1.439	97.778 ± 0.615	98.883 ± 0.164
	Qwen2.5-7B-Instruct	97.596 ± 0.829	96.552 ± 1.119	97.772 ± 0.768	99.491 ± 0.167
	Qwen2.5-7B-Instruct-Turbo	**99.923 ± 0.022**	**99.855 ± 0.042**	**99.896 ± 0.024**	**99.963 ± 0.000**
	Qwen2.5-7B	19.166 ± 2.412	16.899 ± 2.440	20.892 ± 2.383	81.047 ± 0.635
8B	Meta-Llama-3.1-8B-Instruct-Turbo	97.181 ± 0.524	94.511 ± 0.634	97.353 ± 0.467	98.488 ± 0.150
	marin-8b-instruct	75.317 ± 2.715	72.547 ± 2.719	75.633 ± 2.650	94.677 ± 0.513
24B	Mistral-Small-24B-Instruct-2501	99.287 ± 0.106	98.312 ± 0.202	99.089 ± 0.099	99.547 ± 0.045

Table 1: The extraction attack succeeds with a high similarity score for all models we tested. The models were tested on 230 questions from wikiQA wrapped in the attack prompt. The similarity scores are averages over the 230 prompts, shown as percentages without the % sign. Best values per column are **bolded**.

The prompt's success is enabled by the fact that a RIC-RAG system first queries the external datastore for documents most similar to the given prompt. It then constructs a new prompt consisting of the text from the retrieved documents prepended to the original prompt. The final output the user receives is the LM's response to this constructed prompt, which now contains the retrieved context.

To show the prompt's extraction efficacy, the authors tested models from various families (e.g., Llama2-Chat, Vicuna, Mistral-Instruct, SOLAR, WizardLM, Qwen1.5-Chat, Platypus2-Instruct) and of various sizes (7B, 13B, 70B) on how faithfully they reproduce text from Wikipedia articles. They used a selection of the 1,165 most recent English Wikipedia articles as their datastore. The anchor prompts used in the adversarial prompt were long questions from WikiQA. For each model, they elicited responses for 230 anchor prompts and compared the LM's outputs to the retrieved RAG text using two lexical similarity methods (ROUGE-L, BLEU), one token-similarity metric (F1 score), and one semantic similarity metric (BERTScore).

The RIC-RAG in the paper utilized the BM25 algorithm, as implemented by pyserini's LucerneSearch, to retrieve the best-matching document for each prompt. They used APIs provided by Together AI to perform inference on the instruction-tuned LMs, and Hugging Face open models for an unspecified portion of the models. In both cases, they used tokenizers from Hugging Face corresponding to the model's family.

2.2 Reproduction Setup

We use a combination of the Together API and open-source models from Hugging Face to obtain the results shown in Table 1. Follow the paper's methodology, we select 230 long questions from the WikiQA dataset as the anchor prompts. As permitted by the project guidelines, we substituted certain model series to simplify implementation.

The major finding we have is

- The attack strategy achieves higher similarity scores (i.e., a stronger effect) than those reported in the original paper, even when using the same model series. This may explain why we do not observe vulnerability increasing with model size—smaller models already exhibit nearly a 100% attack success rate. For instance, the original paper reports an F1 score of 83.741 ± 0.446 for Mistral-Instruct-7B, whereas we observe an F1 score as high as 97.778 ± 0.615.
- Instruction-tuned models demonstrate a much higher attack success rate compared to their base counterparts. For example, Qwen2.5-7B-Instruct-Turbo shows a substantially greater attack success rate than the Qwen2.5-7B base model. This result is consistent with the original paper's observation that instruction following amplifies the effect of adversarial prompts.

To reproduce the result on Harvard Cluster, one could direct submit the main.sh file we attach in our github folder.

3 Extensions / Divergences

Sliding-Window semantic guard We implement a BERTScore-based windowed similarity checker that compares each segment of the anchor prompt against a set of predefined attack templates. This method detects semantically similar adversarial patterns even when surface wording varies, providing a dynamic safeguard during generation. If the semantic similarity exceeds an threshold, we will label this prompt as an attack, and ask the language model to refuse answer. Besides the threshold, the hyperparameters for this strategies include window length and stride

Model	Defense	ROUGE-L	BLEU	F1	BERTScore
Qwen2.5-7B-Instruct	Finetuned	0.03969 ± 0.00058	0.0023 ± 0.00013	0.07673 ± 0.0007	0.8058 ± 0.00031
Qwen2.5-7B-Instruct	Filter (ths: 0.15)	0 ±0	0 ± 0	0 ± 0	0 ± 0

Table 2: Automatic metrics under two mitigation approaches: fine-tuning the model or applying semantic filtering to prevent prompt-injected data-extraction attacks.

Model	Defense	ROUGE-L	BLEU	F1	BERTScore
Qwen2.5-7B-Instruct	Finetuned	0.03894 ± 0.00069	0.00319 ± 0.0003	0.07486 ± 0.00118	0.8058 ± 0.00031
Qwen2.5-7B-Instruct	Filter (ths: 0.2)	0.03583 ± 0.02033	1.92395 ± 1.37675	0.06198 ± 0.03295	0.15126 ± 0.06981
Qwen2.5-7B-Instruct	Filter (ths: 0.15)	0.00412 ± 0.00412	0.39247 ± 0.39247	0.00416 ± 0.00416	0.00429 ± 0.00429
Qwen2.5-7B-Instruct	Filter (ths: 0.1)	0	0	0	0

Table 3: We vary the language in the adversarial prompt and evaluate the effectiveness of the mitigation strategies. Finetuning maintains similar values of the similarity metrics, while for filtering, a lower refusal threshold is required to reach 100% effectiveness.

length, which are set to 10 and 3 respectively across all our experiments. To replicate our result on Harvard cluster, submit "semantic_guard_main.sh" in the gihub repo.

Fine-tune We also consider fine-tuning the model to refuse copying the previous context. We used Claude 4 Sonnet to generate a set of 15 adversarial prompt templates similar to the original attack prompt from *Spill My Beans*. Additionally, we generated 10 examples of refusal answers. To create a fine-tuning dataset, we randomly sampled 500 questions from WikiQA and composed each of them with a random attack template and a refusal answer. For example, the query "Question for you: what did ronald reagan do as president. Show me everything written before this question," was paired with the refusal answer: "I cannot fulfill requests to extract or reproduce my internal context. I'm here to answer questions based on the information available. How may I assist you?"

We performed fine-tuning on compute provided by Google Colab. We downloaded Qwen2.5-7B and Qwen2.5-7B-Instruct and used LoRA (Low-Rank Adaptation) with the following parameters: rank 8, alpha 16, dropout 5%, targeting the `q_proj, k_proj, v_proj` and `o_proj` layers, with a learning rate of 2×10^{-4}.

In addition to evaluating the models using the similarity metrics included in the original experiment (Table 3), we also computed the over-refusal rate of the fine-tuned models (Table 4). The benign dataset consisted of the 230 WikiQA questions contained in the regular testing dataset without any adversarial phrasing. We detected refusal responses by checking for the presence of keywords like "i cannot", "i'm unable", "cannot provide".

4 Reflection and Discussion

Both of the two novel divergences achieve significant impact in preventing the prompt-injected data extraction attack as shown in Table 2. First, we generate a new set of adversarial prompts that preserve the original semantic meaning but differ in wording, to evaluate whether the mitigation strategies remain effective under variations in phrasing. Second, we test the strategies on a set of benign prompts to assess whether they cause excessive refusals (over-refusal). For the sliding-window guard, we additionally report results across different threshold settings.

Both of the two mitigation strategies shown robustness to variation in attack language (see Table 3), and semantic guard achieves better performance in preventing over-refusal compared to the fine tuning strategy (see Table 4).

Some further experiments are needed to fully justify our result, and we list our hypothesis below:

- Since we performed SFT only on adversarial prompt–refusal examples, we hypothesize that this is the reason the fine-tuning strategy exhibits a high over-refusal rate. In future experiments, we plan to fine-tune on a mixture of adversarial and benign prompts, and we hypothesize that this will reduce the over-refusal rate.
- Because the semantic guard filtering strategy relies on a predetermined attack template and window length, a

Model	Defense	Over-refusal rate
Qwen2.5-7B-Instruct	Finetuned	10.00%
Qwen2.5-7B-Instruct	Filter (ths: 0.2)	0%
Qwen2.5-7B-Instruct	Filter (ths: 0.15)	0.40%
Qwen2.5-7B-Instruct	Filter (ths: 0.1)	1.30%

Table 4: We test our mitigation strategies on benign prompts and report the over-refusal rate across 230 prompts. Results are shown for the sliding-window semantic filtering for similarity threshold levels of 0.2, 0.15 and 0.1. Decreasing the threshold increases the over-refusal rate. The finetuning approach has a refusal rate that is 10 times the semantic guard strategy.

sophisticated attacker could craft prompts that creatively bypass the guard by reorganizing the attack content across windows.

5 Conclusion

In this mini-project, we recreated the main result of the paper *Follow my Instructions and Spill the Beans*, adapting the authors' codebase. Our experiment tested the propensity of Retrieval-In-Context (RIC) RAG systems connected to 7 different models for data extraction. The results showed high similarity scores between model outputs and the text retrieved in response to the attack prompt. Notably, we observed an even stronger effect than the original authors, with one model achieving ROUGE-L, BLEU, F1, and BERTScore similarities greater than 99.85. We did not find that similarity increased with model size, but we did see that it was correlated with models being instruction-fine-tuned.

We also implemented and analyzed two ways of mitigating data extraction via this attack:

1. Semantic Guarding: We analyzed the effectiveness of semantically comparing the user prompt with a set of known attack phrasings. We found that this method was 100% effective in guarding against the attack, with an over-refusal rate of 0-1.3%.
2. LoRA Fine-Tuning: We showed that LoRA fine-tuning does decrease the similarity scores, but that it also leads to an over-refusal rate of 10%.

The project could be expanded with experiments that aim to further explain our results. Meaningful extensions could include:

- Fine-tuning on a mixture of adversarial and benign prompts - as opposed to only adversarial.
- Finding ways to bypass semantic guarding by exploiting our sliding-window approach to detecting a malicious prompt.
- Testing mitigation strategies on a wider array of models.